

Juego CAZANDO

Continuación....

PARTE 3 – MOVER GATO

1. Crear la función *limpiarCanva* (sin parámetros), que limpie toda el área del canvas utilizando `clearRect`.
 2. Crear la función *moverIzquierda* que permita mover el gato **10 píxeles hacia la izquierda**:
 - Restar 10 a la variable *gatoX* y guardar el resultado en la misma variable.
 - Llamar a la función *limpiarCanva*.
 - Llamar a la función *dibujarGato*.
 - Considerar que la comida también se borra al limpiar el canvas → debe volver a dibujarla.
 - Invocar la función *moverIzquierda* desde el botón correspondiente.
- ✓ Probar funcionamiento.
- ✓ Realizar **commit y push**.

-
3. Aplicar la misma lógica para crear:
 - *moverDerecha*
 - *moverArriba*
 - *moverAbajo*
- ✓ Una vez que el gato se mueva en todas las direcciones:
- ✓ Realizar **commit y push**.
-

PARTE 4 – COMER

1. Crear la función *detectarColision* (sin parámetros), que muestre un `alert` cuando el gato toque la comida.
 - Para esto, considerar:
 - Posición **X** e **Y** del gato
 - Posición **X** e **Y** de la comida
 - Dimensiones (**ANCHO** y **ALTO**) de ambos

- Llamar a *detectarColision* cada vez que el gato se mueva.
 - ✓ Probar funcionamiento.
 - ✓ Realizar **commit y push**.
-

2. Cuando el gato toque la comida:

- La comida debe desaparecer y reaparecer en una posición **aleatoria en X y Y**.
 - Utilizar las funciones disponibles en *utilitarios.js* (archivo de recursos).
-

3. Sistema de puntaje:

- Cada vez que el gato coma, gana **1 punto**.
 - Incrementar el puntaje y mostrarlo en pantalla.
 - Utilizar funciones de *utilitarios.js* para actualizar la vista.
 - ✓ Si todo funciona correctamente:
 - ✓ Realizar **commit y push**.
-

PARTE 5 – CUENTA REGRESIVA

1. Crear una variable *tiempo* con valor inicial **10**.
2. Mostrar el tiempo en pantalla:

- Crear un `<span>` para visualizar el valor.
 - Crear la función *restarTiempo* que:
 - Reste 1 a la variable *tiempo*
 - Actualice el valor en pantalla usando *utilitarios.js*
-

3. Usar `setInterval` para ejecutar *restarTiempo* cada segundo, generando un **conteo regresivo**.

- ✓ Probar funcionamiento.
 - ✓ Realizar **commit y push**.
-

4. Condiciones del juego:

- Si el puntaje llega a **6** → mostrar `alert` de ganador y detener el `setInterval`.
 - Si el tiempo llega a **0** → mostrar `alert` de **Game Over** y detener el `setInterval`.
-

PARTE 6 – REINICIAR

1. Agregar un botón **REINICIAR**.
 2. Implementar la funcionalidad necesaria para reiniciar el juego (variables, pantalla, intervalo, etc.).
-

EVIDENCIA

- Adjuntar el **link del repositorio**.
 - Se calificará **cada commit realizado** (historial de trabajo).
-



Evaluación

- Este taller consta de varias partes que se irán incrementando en los siguientes días.
- Se calificará **1 punto por cada commit correcto** realizado en el repositorio remoto.
- Se considerará como commit correcto aquel que:
 - Refleje un avance real del desarrollo
 - Tenga un mensaje claro y descriptivo
 - Mantenga una secuencia lógica del trabajo

